

Tuning ChatGPT Study Mode: Prompt Design for Deep Lecture–Paper Learning

Intro:

This is an attempt to use chatgpt's study mode to comprehensively review lecture content together with the corresponding papers. I understand that chatgpt's study mode has already been explored by other threads, but this study is specifically focused on a prompting style that makes study mode more effective, by carefully designing prompts tailored to how chatgpt study mode is designed. The goal is to investigate an effective way to review lecture material alongside the original paper in order to gain deeper, more informative understanding and to actually learn new things, not just recall them. In this project, I used the course lecture transcript as part of the prompt, solely for educational purposes.

Full Traces:

<https://chatgpt.com/share/69338db6-3cc0-800f-8c1f-e724f2e61d08>

Annotated conversations:

According to OpenAI's description of chatgpt's Study mode (<https://openai.com/index/chatgpt-study-mode/>), the system is designed to behave less like a simple question-answering tool and more like an engaged tutor that guides students through the learning process. A key part of this behavior comes from the underlying system prompts, which structure the interaction to be more dialogic, reflective, and goal-oriented. However, the quality of the assistance still depends heavily on how clearly the student communicates their own needs.

In this context, I have found that the most effective way to use Study mode is to treat the initial prompt as I would an introduction at office hours with a TA. Specifically, I try to provide chatgpt with: (1) what I already know about the topic, (2) what I would like to learn or clarify, and (3) what I am most confused about. By explicitly stating my prior knowledge, learning goals, and points of confusion, I enable Study mode to tailor its questions, explanations, and follow-up prompts much more precisely to my situation.

In the attached fully annotated interaction trace, I illustrate how I applied this strategy in practice when studying a particular lecture and research paper. My hope is that this example will help other students think more deliberately about how they frame their own prompts, and inspire more effective use of chatgpt's Study mode as a learning tool.

I am a little bit confused with the idea of DPO in the 12/2 lecture. My goal is simple: to gain a deeper understanding of DPO, and prepare for the final exam. Here is the initial prompt I used to set up the section:

I'm a graduate-level student in deep learning. I've uploaded a lecture transcript and a research paper about DPO.

Goals for this session:

Build an intuitive and technical understanding of the paper (setup, main ideas, derivations, and experiments). Connect the paper to the concepts in the lecture. Be able to explain the paper to a peer and answer exam-style questions about it.

How I want you to teach me (Study mode rules):

Start by trying to connect the lecture to the paper and identify the specific concept/ideas/formalism that the lecturer put focus on and want me to understand thoroughly. Once You identified all the concepts/ideas, ask me which one I am most not familiar with, and interactively help me understand that concept.

Paper:<https://arxiv.org/abs/2305.18290>

Lecture transcript: So this is the uh welcome everyone back from

My strategy is to 1. Identify clear, actionable goals for this study session, and 2. Specify the mode of studying that is most effective for me personally so chatgpt study mode can tailor the study session.

Cool, this is a perfect pairing: that lecture is *literally* setting you up to understand DPO. Let me first connect the dots, then I'll ask which piece you want to drill into.

1. How the lecture lines up with the DPO paper

From ~15:00 onward, the lecture walks through exactly the stack that DPO is about:

1. RL post-training with verifiable rewards (RLVR)

- Setup: prompts x , many generations y , an **autograder** that gives a numeric reward.
- Objective is something like:

$$\mathbb{E}_{x,y \sim \pi_\theta(\cdot|x)}[R(x,y)]$$

- Use **REINFORCE / log-prob trick**, importance sampling, advantage normalization, etc.

2. KL-regularized RL objective (this is the big conceptual bridge to DPO)

The lecturer writes the RL objective in the RLHF style:

$$\max_{\pi} \mathbb{E}_{x,y \sim \pi}[r(x,y)] - \lambda \text{KL}(\pi(\cdot|x) \parallel \pi_{\text{ref}}(\cdot|x))$$

where π_{ref} is a **reference policy** (e.g., SFT model).

This is *exactly* the formal objective DPO starts from. ar5iv +1

3. RLHF with a learned reward model

- Human gives **pairwise preferences** (winner vs loser).
- Fit a **reward model** $r_\phi(x,y)$ using a Bradley–Terry / logistic preference model.

ar5iv +1

- Then run RL (PPO/GRPO) on:

$$\mathbb{E}_{x,y \sim \pi_\theta}[r_\phi(x,y)] - \lambda \text{KL}(\pi_\theta \parallel \pi_{\text{ref}})$$

4. Problems with naive use of preference data

- “Maximize winners, minimize losers” → catastrophic forgetting / degenerate behavior (the Reddit summary table he showed).
- “Just SFT on winners” → better, but still not ideal, and doesn’t use losers fully.

5. DPO idea (what the paper formalizes)

- The lecture gives the DPO loss in words and rough form:
 - Start from an SFT model as π_{ref} .
 - Use a loss of the form (roughly):

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\underbrace{\beta [\log \pi_{\theta}(y_w|x) - \log \pi_{\text{ref}}(y_w|x)]}_{\text{winner log-ratio}} - \underbrace{\log \pi_{\theta}(y_l|x) - \log \pi_{\text{ref}}(y_l|x)}_{\text{loser log-ratio}} \right) \right].$$

- This is exactly the equation 5/6 in the paper. arXiv +1
- No reward model, no RL loop; just **SGD on preference pairs**.

6. Key theoretical move ("your LM is secretly a reward model")

- Given the KL-regularized RL objective, the paper shows the **optimal policy has a closed form**:

$$\pi^*(y|x) \propto \pi_{\text{ref}}(y|x) \exp(\beta r(x, y)).$$

- Invert this relation: the **reward compatible with a policy** π_{θ} is:

$$r_{\theta}(x, y) = \frac{1}{\beta} (\log \pi_{\theta}(y|x) - \log \pi_{\text{ref}}(y|x)) + \text{const.}$$

- That's the slogan: "your language model is secretly a reward model"—its log-probabilities relative to π_{ref} behave like a reward. arXiv +1

7. TR-DPO / moving reference policy

- Lecture mentions TR-DPO: periodically update π_{ref} to the current π_{θ} (e.g., every 512 steps) → better stability and performance. Oren.ai

8. Experiments (paper vs lecture)

- The paper evaluates on summarization, helpfulness/harmlessness chat tasks, and compares DPO to:
 - SFT
 - RLHF with PPO
 - Other baselines. arXiv +1
- The lecture mostly uses the summarization example (Table 3, "junk" generations) to illustrate why naive "punish losers" fails and why DPO's shaped loss helps.

2. Core concepts/formalisms the lecturer really cares about

If I distill both the lecture and the DPO paper, the "exam-level" concepts are:

1. KL-regularized RLHF objective

- Form: $\max_{\pi} \mathbb{E}[r] - \lambda \text{KL}(\pi \parallel \pi_{\text{ref}})$.
- Why we need the KL term (capability preservation / avoid drifting too far).

2. Mapping between reward and optimal policy

- $\pi^* \propto \pi_{\text{ref}} \exp(\beta r)$.
- Inverse mapping $r_{\theta}(x, y) \sim \log \pi_{\theta} - \log \pi_{\text{ref}}$.
- This is the theoretical heart of "LM as reward model".

3. Bradley-Terry model for human preferences

- How pairwise preferences (x, y_w, y_l) induce a likelihood:

$$P(y_w \succ y_l \mid x) = \sigma(\beta (r(x, y_w) - r(x, y_l))).$$

4. DPO loss derivation & intuition

- Plug the "secret reward" r_{θ} into the Bradley-Terry model → get the DPO loss.
- How gradients end up being "increase winner prob, decrease loser prob" with a **sigmoid-based weight** that avoids over-punishing.

5. Why naive winner/loser training breaks SFT behavior

- Catastrophic forgetting + ambiguous negative signal ("don't be like this entire sentence").

6. Practical differences: DPO vs RLHF (PPO/GRPO)

- Offline vs on-policy.
- No reward model, no rollout loop, fewer hyperparameters; but still tied to KL-regularized RLHF objective.

Note that chatgpt study mode is able to follow the initial instruction quite well. I can strongly feel that chatgpt becomes more “collaborative” and “engaging”. I like how it is making meaningful connections between the lecture and the paper, and it also analyzes this from the instructor's perspective: it pays special attention to how the idea is being built up and presented to students. I think this is crucial for chatgpt to understand the focus of this study session.

3. Your turn: what should we zoom in on?

Let's pick one concept to really unpack *first*, then build around it.

Which of these are you **least** comfortable with right now?

1. The KL-regularized RLHF objective and why its solution has the Boltzmann form $\pi^* \propto \pi_{\text{ref}} \exp(\beta r)$.
2. The “LM is secretly a reward model” mapping $r_\theta(x, y) \sim \log \pi_\theta - \log \pi_{\text{ref}}$.
3. The derivation of the DPO loss from the Bradley–Terry preference model.
4. The gradient behavior / intuition of the DPO loss (why it doesn't nuke SFT behavior like the naive baselines).
5. The big-picture pipeline comparison: RLHF vs DPO vs TR-DPO (what changes in practice, pros/cons).

Pick **one number** (or write your own variant), and we'll go step-by-step on that topic first.

    ...   Sources

One thing I don't like about this answer is that these concepts seem too abstract (too “fancy”), and it is hard for me to understand the intended intuitions behind them. The reason behind this is Chatgpt is referring to phrases being used in the paper – for students who have not read the original paper, the wording may seem a bit abstract. A better prompt in this case would be to instruct Chatgpt to assume the student has not read the original paper yet.

I asked something that is not within the list to see how Chatgpt would respond ;

I feel like I don't understand the intuition and math formula for the loss function in DPO

Response:

Thought for 25s >

Nice, this is the heart of the paper, so if we nail this you'll feel much more in control of DPO.

Let's build the loss in **two small steps**:

1. A probabilistic model of **human preferences**
2. How that turns into the **DPO loss formula** with log-probabilities

I'll keep the algebra light but precise.

1. Model of human preferences (Bradley–Terry)

We have triples (x, y_w, y_l) :

- x : prompt
- y_w : **winner** response (human preferred)
- y_l : **loser** response

Assume there is an underlying **reward function** $r(x, y)$ that humans are “noisily” following when they choose between two answers.

A super standard assumption (Bradley–Terry / logistic model) is:

$$P(\text{human prefers } y_w \text{ over } y_l \mid x) = \sigma(\beta [r(x, y_w) - r(x, y_l)])$$

where:

- $\sigma(z) = \frac{1}{1+e^{-z}}$ is the **sigmoid**
- $\beta > 0$ controls how “sharp” the preferences are: large $\beta \approx$ deterministic, small $\beta \approx$ noisy

Then the **negative log-likelihood** loss for one pair is:

$$\mathcal{L}_{\text{pair}}(r) = -\log P(y_w \succ y_l \mid x) = -\log \sigma(\beta [r(x, y_w) - r(x, y_l)])$$

So far, this is just:

“We observed that the human chose y_w over y_l ; penalize the model if it thinks that’s unlikely.”

2. "Your LM is secretly a reward model"

Now DPO's key trick: **don't learn a separate reward model** r_ϕ . Instead, *define* the reward in terms of the policy π_θ .

From the KL-regularized RLHF objective, the optimal policy under a reward r and reference policy π_{ref} has the form:

$$\pi^*(y|x) \propto \pi_{\text{ref}}(y|x) \exp(\beta r(x, y))$$

If you **solve this for r** , you get (up to an additive constant):

$$r(x, y) \propto \frac{1}{\beta} [\log \pi^*(y|x) - \log \pi_{\text{ref}}(y|x)]$$

DPO says: *let's treat our current model π_θ as if it is this optimal policy*. So define

$$r_\theta(x, y) = \frac{1}{\beta} [\log \pi_\theta(y|x) - \log \pi_{\text{ref}}(y|x)]$$

(You can ignore the constant; it cancels in differences.)

Now plug this r_θ into the preference model from step 1:

$$P_\theta(y_w \succ y_l | x) = \sigma(\beta[r_\theta(x, y_w) - r_\theta(x, y_l)])$$

Substitute the expression for r_θ :

$$\beta[r_\theta(x, y_w) - r_\theta(x, y_l)] = \log \pi_\theta(y_w|x) - \log \pi_{\text{ref}}(y_w|x) - (\log \pi_\theta(y_l|x) - \log \pi_{\text{ref}}(y_l|x))$$

Call that whole thing $D_\theta(x, y_w, y_l)$. Then:

$$P_\theta(y_w \succ y_l | x) = \sigma(D_\theta(x, y_w, y_l))$$

So the DPO loss for one pair is simply:

$$\mathcal{L}_{\text{DPO}}(\theta; x, y_w, y_l) = -\log \sigma\left(\underbrace{\log \pi_\theta(y_w|x) - \log \pi_{\text{ref}}(y_w|x)}_{\text{winner log-ratio}} - \underbrace{\log \pi_\theta(y_l|x) - \log \pi_{\text{ref}}(y_l|x)}_{\text{loser log-ratio}}\right)$$

And the overall loss is the expectation/average over all preference pairs in the dataset.

Intuition in words:

- The **log-ratio** $\log \pi_\theta - \log \pi_{\text{ref}}$ says:
 | "How much more (or less) does the new model like this answer than the SFT baseline?"
- The inner difference compares **winner vs loser**:
 | "Is the new model boosting the winner relative to the loser (more than the reference did)?"
- The **sigmoid** $+\log$ is just **logistic regression**:
 | "We want the probability that the model prefers the human winner to be high $\rightarrow$ big positive argument $\rightarrow$ small loss."

So the DPO loss is nothing exotic: it's **logistic loss on pairwise preferences**, where the feature is "log-prob gap between winner and loser, adjusted relative to a reference model."

3. Quick check

The key conceptual step is:

DPO loss = negative log of "probability that the model's implied reward says winner better than loser."

Does it make sense to you that:

- We model $P(y_w \succ y_l)$ with a **sigmoid of a reward difference**, and
- We then **plug in** $r_\theta(x, y) = \frac{1}{\beta}(\log \pi_\theta - \log \pi_{\text{ref}})$ to get the log-prob terms in the loss?

If something still feels fuzzy, which part is the main sticking point for you right now:

- A. Why the logistic/sigmoid shows up at all
- B. Why the "reward" can be written as $\log \pi_\theta - \log \pi_{\text{ref}}$
- C. How the winner/loser difference turns into that nested subtraction inside the loss

📄 👍 🗨️ ↻ ...

I like that the study mode is trying to build up ideas first with intuitions and then derive the formula. (Just like in lectures). Whether this is an innate behavior of the study mode or is influenced by the prompt is an interesting follow up question to investigate. I also like that instead of dumping long and complex math, it tries to keep the algebra light at first, with the possibility of diving deeper later. From a learner's perspective, this makes more sense to me as I won't be able to understand all the math the second I start learning. I need to first build up intuitions, and then use the intuitions to break down the hard math. I think this is a feature from the study mode itself instead of a result of the prompting, but it may be a good thing to add to future prompts (to reinforce this helpful behavior).

Note that instead of simply asking if I am still confused, study mode compiles "common confusions" for me. Personally, this is extremely helpful because when navigating with a new and big concept, I sometimes "feel" like I understand, but there's often missing details I don't even realize that I haven't fully understood. By providing this mental checkpoint lists, I can cross them off one at a time in my mind. This not only helped me reflect on my own understanding, but also helped me organize my thoughts.

can you explain B in more detail?

Nice, this is exactly the right place to dig in. Let's zoom in on why we can write

$$r_\theta(x, y) \propto \log \pi_\theta(y|x) - \log \pi_{\text{ref}}(y|x)$$

I'll walk you through the logic step by step, and I'll ask you one small check in the middle.

Step 1: Start from the KL-regularized RL objective

The lecture already wrote this (and the paper does too). The "ideal" post-training objective is:

$$\max_{\pi} \mathbb{E}_{x, y \sim \pi(\cdot|x)} [r(x, y)] - \lambda \text{KL}(\pi(\cdot|x) \parallel \pi_{\text{ref}}(\cdot|x))$$

For a *fixed* prompt x , you can think of this as an optimization problem over a probability distribution $\pi(\cdot|x)$ on responses y .

For that fixed x , write the objective as:

$$J(\pi) = \sum_y \pi(y|x) r(x, y) - \lambda \sum_y \pi(y|x) \log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)}$$

Plus the constraint $\sum_y \pi(y|x) = 1$.

This is a *pure calculus* problem: maximize a linear term in π minus a KL term.

Step 2: Solve for the optimal policy π^*

Introduce a Lagrange multiplier α for the normalization constraint:

$$\mathcal{L}(\pi, \alpha) = \sum_y \pi(y|x) r(x, y) - \lambda \sum_y \pi(y|x) \log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} + \alpha \left(\sum_y \pi(y|x) - 1 \right)$$

Take derivative w.r.t. $\pi(y|x)$ and set to zero:

$$\frac{\partial \mathcal{L}}{\partial \pi(y|x)} = r(x, y) - \lambda \left[\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} + 1 \right] + \alpha = 0$$

Rearrange for the log-ratio:

$$\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} = \frac{1}{\lambda} (r(x, y) + \alpha - \lambda)$$

Exponentiate:

$$\frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} \propto \exp \left(\frac{1}{\lambda} r(x, y) \right)$$

Absorb constants into a normalization term $Z(x)$, and set $\beta = 1/\lambda$:

$$\pi^*(y|x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y|x) \exp(\beta r(x, y))$$

This is the Boltzmann / Gibbs form you've seen: reference policy times an exponential of reward.

So far:

Given reward r , the optimal policy π^* has this closed form.

Step 3: Invert the relation: solve for reward in terms of policy

Now do the opposite: **assume** your policy π_θ has this form for *some* reward function. Then you can solve for that reward.

From:

$$\pi_\theta(y|x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y|x) \exp(\beta r_\theta(x, y))$$

take logs:

$$\log \pi_\theta(y|x) = \log \pi_{\text{ref}}(y|x) + \beta r_\theta(x, y) - \log Z(x)$$

Rearrange for r_θ :

$$\beta r_\theta(x, y) = \log \pi_\theta(y|x) - \log \pi_{\text{ref}}(y|x) + \log Z(x)$$

So

$$r_\theta(x, y) = \frac{1}{\beta} (\log \pi_\theta(y|x) - \log \pi_{\text{ref}}(y|x)) + \underbrace{\frac{1}{\beta} \log Z(x)}_{\text{constant in } y}$$

Key point: **for a fixed prompt x** , $\log Z(x)$ does *not* depend on y . So:

- When you take **differences** $r_\theta(x, y_w) - r_\theta(x, y_l)$, that constant cancels.
- When you feed those differences into the sigmoid preference model, the constant doesn't matter.

So for preferences, we can effectively treat:

$$r_\theta(x, y) \propto \log \pi_\theta(y|x) - \log \pi_{\text{ref}}(y|x)$$

That's the "your LM is secretly a reward model" line: its **log-probs relative to the reference** behave like a reward function.

Step 4: Why this is the *right* reward, conceptually

Intuitively, this reward says:

- If $\pi_\theta(y|x)$ is **larger** than $\pi_{\text{ref}}(y|x)$, then $\log \pi_\theta - \log \pi_{\text{ref}} > 0 \rightarrow$ **positive reward shift** $\rightarrow$ the new policy “likes this answer more” than the reference.
- If it’s **smaller**, the log-ratio is negative $\rightarrow$ **negative reward shift**.

So:

DPO interprets “how much more than the SFT model does π_θ like this answer?” as “how good is this answer?”

And we justified this by **deriving** it from the KL-regularized RL objective, not just hand-waving.

Quick check for you

Can you say in your own words why the **constant** $\log Z(x)$ doesn’t matter for DPO?

Hint: think about where $r_\theta(x, y_w) - r_\theta(x, y_l)$ appears in the DPO loss.

📄 👍 💬 ↺ ...

Another nice feature of study mode is that ChatGPT will ask small quiz questions to check my understanding. I find this very helpful. However, one thing I’ve noticed is that, although these questions deepen my understanding of the concepts (for example, the derivation of the loss function), they sometimes feel somewhat “random”: the question may not correspond directly to the original question I started with (why the “reward” can be written as $\log \pi_\theta - \log \pi_{\text{ref}}$). My main concern is that, because the questions asked by ChatGPT—and my answers to them—shape how the conversation unfolds, questions that are not closely related to the overarching big question can unintentionally steer the study session in a different direction. Thus, I think another nice thing to mention in the prompt is to ask small quiz questions that correspond directly to the detailed learning objectives.

i can't. Can you explain further?

Thought for a couple of seconds >

Totally fair — this is one of those “obvious once you see it” things. Let’s slow it down and make it concrete.

1. Where does the reward appear in DPO?

Remember the **pre-DPO** version of the loss, written in terms of some reward r :

$$\mathcal{L}_{\text{pair}}(r) = -\log \sigma(\beta[r(x, y_w) - r(x, y_l)])$$

So the only place r appears is **inside a difference**:

$$r(x, y_w) - r(x, y_l)$$

That’s the *only* combination of rewards the loss ever sees.

2. Now add a constant to the reward

Suppose instead of r we use

$$r'(x, y) = r(x, y) + C(x)$$

where $C(x)$ is **any function that depends only on the prompt x** (like $\frac{1}{\beta} \log Z(x)$ from the derivation).

Compute the difference that actually appears in the loss:

$$\begin{aligned} r'(x, y_w) - r'(x, y_l) &= (r(x, y_w) + C(x)) - (r(x, y_l) + C(x)) \\ &= r(x, y_w) - r(x, y_l) + C(x) - C(x) \\ &= r(x, y_w) - r(x, y_l) \end{aligned}$$

So the **constant cancels exactly** in the difference.

That means:

$$\beta[r'(x, y_w) - r'(x, y_l)] = \beta[r(x, y_w) - r(x, y_l)]$$

so

$$\sigma(\beta[r'(w) - r'(l)]) = \sigma(\beta[r(w) - r(l)])$$

and therefore

$$\mathcal{L}_{\text{pair}}(r') = \mathcal{L}_{\text{pair}}(r)$$

Conclusion:

If you add any constant $C(x)$ to all rewards for a given prompt, the DPO pair loss is *literally unchanged* for that pair.

3. Intuition (no equations)

Think of a simpler analogy:

- You have two answers with “scores” 3 and 1.
- A human prefers the higher score ($3 > 1$).
- If you add +10 to both scores, you get 13 and 11.
- The preference **doesn’t change**: $13 > 11$.

What matters for preferences is the **gap** between them:

- Original gap: $3 - 1 = 2$
- Shifted gap: $13 - 11 = 2$

Same gap → same “strength” of preference.

DPO is basically doing **logistic regression on that gap**. Adding a constant to both terms doesn’t change the gap, so it doesn’t change:

- The predicted probability that the model prefers y_w
- The loss
- Or the gradient.

That’s why that $\frac{1}{\beta} \log Z(x)$ term from the derivation is harmless: it’s just a **per-prompt offset** that vanishes in the winner–loser difference.

4. How this ties back to the “secret reward” formula

From before, we had:

$$r_{\theta}(x, y) = \frac{1}{\beta} (\log \pi_{\theta}(y|x) - \log \pi_{\text{ref}}(y|x)) + \underbrace{\frac{1}{\beta} \log Z(x)}_{\text{constant in } y}$$

DPO only ever uses $r_{\theta}(x, y_w) - r_{\theta}(x, y_l)$, so the $\frac{1}{\beta} \log Z(x)$ part cancels out. That’s why people usually just write

$$r_{\theta}(x, y) \propto \log \pi_{\theta}(y|x) - \log \pi_{\text{ref}}(y|x)$$

and silently drop the constant.

If you’re okay with that now, next nice step is to look at what the **gradient** of the DPO loss looks like (and why it’s “increase winner prob, decrease loser prob, but with a sigmoid weighting”).

Do you want to go there next, or would you rather first re-derive the full DPO loss once yourself with me checking each step?

📄 👍 🗨️ ↺ ...

Why does the original loss have a sigmoid?

Thought for a few seconds >

Nice, this is a key “why” question, not just a “memorize the formula” one.

Short answer:

The sigmoid shows up because DPO is doing **logistic regression on pairwise preferences**.

It comes from a probabilistic model of “what is the probability that a human prefers y_w over y_l given their (latent) rewards?”

Let’s build that up slowly.

1. What are we trying to model?

For each triple (x, y_w, y_l) :

- Human saw two answers y_w (winner) and y_l (loser) for the same prompt x .
- Human chose y_w .

We want a model that can say:

$$P(\text{human picks } y_w \text{ over } y_l \mid x, \text{model parameters})$$

and then we’ll do **maximum likelihood** on these choices.

So we need a **parametric form** for this probability.

2. Basic requirement: depend only on *reward difference*

Intuitively, humans are more likely to pick y_w over y_l if:

- $r(x, y_w)$ is high,
- $r(x, y_l)$ is low,
- and what really matters is the **gap** $r(x, y_w) - r(x, y_l)$.

So any reasonable model should have the structure:

$$P(y_w \succ y_l \mid x) = f\left(r(x, y_w) - r(x, y_l)\right)$$

for some increasing function $f: \mathbb{R} \rightarrow (0, 1)$.

We want:

- If $r_w \gg r_l$: probability ≈ 1
- If $r_w \ll r_l$: probability ≈ 0
- If $r_w = r_l$: probability $= 0.5$

That's exactly the shape of... the **sigmoid**.

3. The Bradley–Terry / logistic choice model

This is a classic model in statistics and choice theory:

- Each item y has a "score" $s(y)$.
- The probability that item i beats item j is

$$P(i \succ j) = \sigma(s(i) - s(j))$$

with $\sigma(z) = \frac{1}{1+e^{-z}}$.

For DPO we use scores based on reward:

$$s(x, y) = \beta r(x, y)$$

So:

$$P(y_w \succ y_l \mid x) = \sigma(\beta[r(x, y_w) - r(x, y_l)])$$

This is *exactly* the form in the paper. The β just controls "how deterministic" human choices are:

- $\beta \rightarrow \infty$: always pick higher reward (hard argmax)
 - $\beta \rightarrow 0$: almost random (no preference)
-

In study mode, ChatGPT tends to approach problems from an "intuition first $\rightarrow$ prove it" perspective, which I find very helpful. It usually starts by building a high-level, conceptual understanding of the idea before moving on to the more formal derivations or proofs. This structure makes it easier for me to see *why* something is true before getting into how to show it rigorously. Especially for topics like loss function derivations or RL algorithms, this step-by-step progression from intuition to formal proof helps the material feel much more natural and easier to remember.

Since the rest of the conversation is simply me asking different questions, there aren't many "study mode" features to comment on, I thus omitted the rest of the conversation in this annotated trace. Please refer to the full conversation for the rest of the QAs.

Summary:

In this attempt, I used a case study to explore how to better prompt chatgpt's study mode in order to have a more effective study session. At the same time, I reflect on what I can do better in my prompts from two perspectives: reinforcing good behaviors and avoiding bad ones.

I discovered some nice features of chatgpt's study mode that can be reinforced in the prompt to encourage good behaviors and reduce unhelpful ones. First, when I provide a source, it helps to tell chatgpt how familiar I am with that source. Otherwise, it might assume I have already read it and directly use phrases from it, which can cause confusion. Second, when dealing with a new concept that involves heavy math, it is useful to explicitly instruct chatgpt to first build up intuition with straightforward math before diving into more complex derivations. This makes it easier for my brain to understand the complex math. (Although chatgpt's study mode implicitly does this, it is still helpful to reinforce it in the prompt.)

Third, another (partly implicit) behavior that is nice to reinforce is asking chatgpt to provide "mental checkpoints" when checking whether I am confused—for example, bullet points I can mentally cross off to help me organize my thoughts and reflect on whether I really understand the concept. Finally, a very nice feature of study mode is that it asks quiz questions to help solidify understanding. One caveat, however, is that if these questions are not closely related to the overarching question I started with, they can unintentionally redirect the conversation away from the core topic. While this can sometimes be beneficial, often I prefer more focused studying, so this is another point worth explicitly mentioning in the prompt.

Overall, this case study gave me a clearer sense of how to "co-pilot" chatgpt's study mode so that it better matches my learning goals and supports more focused, effective studying.